# 上機考題目索引

看題目 → 找到對應資料夾 → 打開 `practice/資料夾/` 裡的程式碼。藍色 LabN_ex = 課堂 Exercise 原題;綠色 LabN_gK = 猜題(g1 最可能)。每個資料夾的 `README.md` 有完整解說與易錯點。

| 資料夾 | 題目 | 可能性 | 檔案 |
|---|---|---|---|
| Lab 1:Shell Script | | | |
| `Lab1_ex` | 猜數字遊戲 Number Guessing Game | 原題 | ex.sh |
| `Lab1_g1` | 【原題重現】Number Guessing Game（1~10，範圍逐步縮小） | 高 | NGG.sh |
| `Lab1_g2` | 【加料版猜數字】範圍與次數改用參數給，並統計猜了幾次 | 高 | guess.sh |
| `Lab1_g3` | 【case + function + expr】選單式計算機 | 中高 | calc.sh |
| `Lab1_g4` | 【Pipe】兩支 script 串接：產生分數 \| 判定等第並統計 | 中 | gen.sh, grade.sh |
| `Lab1_g5` | 【命令列參數 + expr】三個數字：加總、最大值、階乘 | 中 | nums.sh |
| `Lab1_g6` | 【字串比較】帳號密碼檢查（最多 3 次） | 中低 | login.sh |
| Lab 2:Adding a New System Call | | | |
| `Lab2_ex1` | Exercise-1(60 分):完成範例,新增 hello syscall | 原題 | hello.c, test.c |
| `Lab2_ex2` | Exercise-2(40 分):SYSCALL_DEFINE1 印 n 次 hello | 原題 | hello.c, test.c |
| `Lab2_g1` | Exercise 2 原題:傳入 n,印 n 次 hello | 高 | hello.c, test.c |
| `Lab2_g2` | SYSCALL_DEFINE2:兩個參數相加,並回傳結果 | 高 | hello.c, test.c |
| `Lab2_g3` | 用回傳值把計算結果帶回 user space | 中~高 | hello.c, test.c |
| `Lab2_g4` | 流程/填空/口試題:新增 syscall 要改哪些檔案 | 中 | |
| `Lab2_g5` | 新增第二個 syscall(編號 463):兩數相加 | 中低 | hello.c, test.c |
| `Lab2_g6` | 參數檢查:不合法時回傳錯誤 | 低 | hello.c, test.c |
| Lab 3:System Call vs APIs | | | |
| `Lab3_ex` | 只用 <unistd.h> 印出 integer | 原題 | ex, ex.c |
| `Lab3_g1` | 只用 <unistd.h> 印出字串與整數陣列（含負數） | 高 | q1.c |
| `Lab3_g2` | 只用 <unistd.h> 從 stdin（fd 0）讀入兩個整數，輸出和 | 高 | q2.c |
| `Lab3_g3` | 只用 system call 複製檔案（自製 cp） | 中高 | q3.c |
| `Lab3_g4` | 用 open() 指定權限建立檔案，把整數寫進檔案 | 中 | q4.c |
| `Lab3_g5` | write() vs fwrite() 一次寫 1 byte，比較時間並用 strace 觀察 | 中 | q5_fwrite.c, q5_write.c |
| `Lab3_g6` | 用 C library（fopen/fread/fwrite/fclose）改寫檔案複製 | 低 | q6.c |
| Lab 4:Library | | | |
| `Lab4_ex` | Exercise 1 static library + Exercise 2 shared library(sort) | 原題 | ex1, ex1.c, ex_dy, exlib.a, exlib.h, exlib.so, sort.c, sort.o |
| `Lab4_g1` | 降冪排序 library（10 個不重複亂數） | 高 | exercise.c, exlib.h, sort.c |
| `Lab4_g2` | 統計函式庫 max / min / average（多個 .o 包成一個 library） | 高 | average.c, main.c, max.c, min.c, mylib.h |
| `Lab4_g3` | 原題重現 + ldd 驗證 + 刪除 library 實驗 | 中高 | exercise.c, exlib.h, sort.c |

| | | | |
|---|---|---|---|
| Lab4_g4 | 更新 library 後觀察 static / dynamic 行為差異 | 中 | exercise.c, exlib.h, sort_v1.c, sort_v2.c |
| Lab4_g5 | 擴充投影片範例 — 四則運算 library | 中 | add.c, div.c, lib.h, main.c, mul.c, sub.c |
| Lab4_g6 | 字串函式庫 reverse + to_upper | 低 | main.c, reverse.c, strlib.h, upper.c |
| Lab 5:Kernel Module | | | |
| Lab5_ex | date2timestamp + myage | 原題 | Makefile, Module.symvers, d2ts.c, d2ts.ko, d2ts.mod, d2ts.mod.c, d2ts.mod.o, d2ts.o, modules.order, myage.c, myage.ko, myage.mod, myage.mod.c, myage.mod.o, myage.o |
| Lab5_g1 | 倒數計時 | 高 | Makefile, countdown.c, deadline.c |
| Lab5_g2 | 加上時分秒：活了幾年幾天幾時幾分幾秒 | 高 | Makefile, date2timestamp.c, myage.c |
| Lab5_g3 | Export 函式：兩個日期相差幾天 | 中高 | Makefile, datelib.c, daydiff.c |
| Lab5_g4 | 精確足歲 + 下次生日倒數 | 中 | Makefile, birthday.c, date2timestamp.c |
| Lab5_g5 | 出生那天是星期幾 | 中低 | Makefile, date2timestamp.c, weekday.c |
| Lab5_g6 | 單一 module：該年第幾天、是否閏年 | 低 | Makefile, dayofyear.c |
| Lab 6:Process Control | | | |
| Lab6_ex1 | Exercise 1(40 分):fork 後 global / local 變數 | 原題 | ex1, ex1.c |
| Lab6_ex2 | Exercise 2(50 分):5 個 process 的樹 | 原題 | ex2, ex2.c |
| Lab6_g1 | 換一棵 process tree（6 個 process） | 高 | q1_tree.c |
| Lab6_g2 | Ex1 改成 parent 和 child 都遞增，加上 heap 變數 | 高 | q2_vars.c |
| Lab6_g3 | fork–exec，child 用 execlp 執行 ls -l，parent 等待後印訊息 | 中高 | q3_exec.c |
| Lab6_g4 | 用迴圈 fork N 個 child（扇形樹），parent wait N 次並印出回收的 child PID | 中 | q4_fan.c |
| Lab6_g5 | 鏈狀 process tree A → B → C → D | 中 | q5_chain.c |
| Lab6_g6 | fork() 連續呼叫 n 次，總共有幾個 process？ | 低～中 | q6_count.c |
| Lab 7:IPC(Message Queue / Shared Memory) | | | |
| Lab7_ex1 | Exercise-Lab1(50 分):Message Queue sender / receiver | 原題 | receiver, receiver.c, sender, sender.c |
| Lab7_ex2 | Exercise 2(50 分):Shared Memory producer / consumer | 原題 | consumer, consumer.c, producer, producer.c |
| Lab7_g1 | 雙向 Message Queue：計算機 client / server | 高 | client.c, server.c |
| Lab7_g2 | Shared Memory 寫入多筆訊息直到 exit | 高 | consumer.c, producer.c |

| | | | |
|---|---|---|---|
| Lab7_g3 | Shared Memory 傳整數陣列，consumer 計算 | 中高 | consumer.c, producer.c |
| Lab7_g4 | 依 Priority 接收 + mq_getattr + O_NONBLOCK | 中 | receiver.c, sender.c |
| Lab7_g5 | 結合兩者：shm 放資料、message queue 當通知 | 中 | consumer.c, producer.c |
| Lab7_g6 | 驗證觀念：kernel persistence 與 MAP_PRIVATE vs MAP_SHARED | 中低 | cleaner.c, reader.c, writer.c |

# Lab 1:Shell Script

## Lab1_ex 猜數字遊戲 Number Guessing Game 課堂 Exercise 可能性:原題

檔案:ex.sh · 出處:簡報 p.48–50

- 數字範圍定為 1~10。
- 一開始請使用者輸入目標數字。
- 不斷縮小範圍,直到玩家猜中目標數字。
- 要用 echo、read、loop、conditionals,並用 function 詢問猜測的數字。

## Lab1_g1 【原題重現】Number Guessing Game（1~10，範圍逐步縮小） 猜題 可能性:高

檔案:NGG.sh · 出處:review/predict/Lab1_predict.md

寫 NGG.sh：

1. 先讓使用者輸入目標數字（1~10），超出範圍要重新輸入。
2. 用 function 詢問猜測值，提示目前的範圍。
3. 猜太大印 Too high!，範圍上限改成 guess-1。猜太小印 Too low，下限改成 guess+1。猜中印 Bingo! 並結束。
4. 必須用到 echo、read、loop、conditionals、function。

```
$ ./NGG.sh
Please enter the target number between 1 and 10:
5
Please enter a number between 1 and 10
8
Too high!
Please enter a number between 1 and 7
2
Too low
Please enter a number between 3 and 7
...
Bingo!
```

## Lab1_g2 【加料版猜數字】範圍與次數改用參數給，並統計猜了幾次 猜題 可能性:高

檔案:guess.sh · 出處:review/predict/Lab1_predict.md

寫 guess.sh，用法是 ./guess.sh min max limit：

1. 如果參數不是 3 個，印出 Usage: ./guess.sh min max limit（要用 $0）然後結束。
2. 輸入目標數字後，最多讓玩家猜 limit 次，每次提示 [第幾次/limit] 跟目前的範圍（用 function）。
3. 猜中時印 Bingo! You got it in N tries.。次數用完時印 Game over! The answer is X.。

```
$ ./guess.sh 1 100 5
Please enter the target number between 1 and 100:
42
[1/5] Please enter a number between 1 and 100
50
Too high!
[2/5] Please enter a number between 1 and 49
25
Too low
[3/5] Please enter a number between 26 and 49
42
Bingo! You got it in 3 tries.
```

## Lab1_g3 【case + function + expr】選單式計算機 猜題 可能性:中高

檔案:calc.sh · 出處:review/predict/Lab1_predict.md

寫 calc.sh：

1. 用 function show_menu 印出選單：a) add s) sub m) mul d) div q) quit。
2. 讀入選項，用 case 分支。a/s/m/d 要再讀「兩個數字」（同一行，用 read x y），然後呼叫 function calc 印出結果。
3. 除數是 0 時印 Error: divide by zero。輸入不認識的選項時印 Invalid option。輸入 q 時印 Bye! 並結束。
4. 一定要用 expr 計算。

```
$ ./calc.sh
===== Calculator =====
a) add  s) sub  m) mul  d) div  q) quit
Choose:
m
Enter two numbers:
4 6
4 * 6 = 24
...
Choose:
q
Bye!
```

## Lab1_g4 【Pipe】兩支 script 串接：產生分數 | 判定等第並統計 猜題 可能性:中

檔案:`gen.sh` `grade.sh` · 出處:review/predict/Lab1_predict.md

1. `gen.sh`：用 for 迴圈依序輸出 `95 82 60 45 73`，每行一個（仿 ex3.sh）。
2. `grade.sh`：用 `while read` 從標準輸入一行一行讀分數（仿 ex4.sh），印出 `分數 -> 等第`（>=90 A、>=80 B、>=60 C、其他 F）。讀完後印出筆數、及格人數、平均（整數）。
3. 執行 `./gen.sh | ./grade.sh`。

```
$ ./gen.sh | ./grade.sh
95 -> A
82 -> B
60 -> C
45 -> F
73 -> C
count=5 pass=4 avg=71
```

## Lab1_g5 【命令列參數 + expr】三個數字：加總、最大值、階乘 猜題 可能性:中

檔案:`nums.sh` · 出處:review/predict/Lab1_predict.md

寫 `nums.sh`，用法 `./nums.sh n1 n2 n3`：

1. 第一行印 `Welcome to use $0`。參數不是 3 個時印 `Usage: ...` 並結束。
2. 印出參數個數、三個數的和、最大值。
3. 用 function `fact` 搭配 while 迴圈，計算 `$1` 的階乘。

```
$ ./nums.sh 5 3 9
Welcome to use ./nums.sh
Hi, we have 3 argument(s): 5 3 9
sum = 17
max = 9
5! = 120
```

## Lab1_g6 【字串比較】帳號密碼檢查（最多 3 次） 猜題 可能性:中低

檔案:login.sh · 出處:review/predict/Lab1_predict.md

寫 login.sh：帳號 admin、密碼 1234。每次在同一行讀入「帳號 密碼」。

- 有一個是空的：印 Username and password must not be empty
- 帳號錯：印 No such user: xxx
- 密碼錯：印 Wrong password
- 成功：印 Welcome, admin! (tries: N) 並結束
- 失敗 3 次：印 Too many attempts, locked.

```
$ ./login.sh
Enter username and password:
root 1
No such user: root
Enter username and password:
admin 1234
Welcome, admin! (tries: 2)
```

## Lab 2:Adding a New System Call

### Lab2_ex1 Exercise-1(60 分):完成範例,新增 hello syscall 課堂 Exercise 可能性:原題

檔案:hello.c test.c · 出處:簡報 p.41

照簡報步驟新增 sys_hello(SYSCALL_DEFINE0,印出 Hello),編譯安裝 kernel,用 syscall(462) 測試並找助教 demo。

要改的檔案:

- ~/linux/hello/hello.c、~/linux/hello/Makefile(obj-y += hello.o)
- ~/linux/include/linux/syscalls.h:加 asmlinkage long sys_hello(void);
- ~/linux/Kbuild:加 obj-y += hello/
- ~/linux/arch/x86/entry/syscalls/syscall_64.tbl:加 462 common hello sys_hello

### Lab2_ex2 Exercise-2(40 分):SYSCALL_DEFINE1 印 n 次 hello 課堂 Exercise 可能性:原題

檔案:hello.c test.c · 出處:簡報 p.42–43

改寫範例的 hello.c,使用 SYSCALL_DEFINE1(),讓 syscall 可接受一個參數,參數 = 輸出 hello 的次數。例:syscall(462, 3) 即為輸出 3 次 hello。改好後要重新 make -j $(nproc)、sudo make -j $(nproc) modules_install、sudo make install、reboot。

### Lab2_g1 Exercise 2 原題:傳入 n,印 n 次 hello 猜題 可能性:高

檔案:hello.c test.c · 出處:review/predict/Lab2_predict.md

題目:改寫範例的 hello.c,用 SYSCALL_DEFINE1() 讓 syscall 接收一個參數 n,印出 n 次 hello。執行 syscall(462, 3)。

預期輸出

```
$ ./test
0
$ sudo dmesg | tail
[  554.662505] hello
[  554.662509] hello
[  554.662510] hello
```

## Lab2_g2 SYSCALL_DEFINE2:兩個參數相加,並回傳結果 猜題 可能性:高

檔案:`hello.c` `test.c` · 出處:review/predict/Lab2_predict.md

題目:改寫 hello syscall,用 `SYSCALL_DEFINE2()` 接收兩個整數 a、b。在 kernel log 印出 `a + b = 結果`,並把結果回傳給 user program 印出。執行 `syscall(462, 3, 5)`。

預期輸出

```
$ ./test
return = 8
$ sudo dmesg | tail -1
[  xxx.xxxxxx] 3 + 5 = 8
```

## Lab2_g3 用回傳值把計算結果帶回 user space 猜題 可能性:中~高

檔案:`hello.c` `test.c` · 出處:review/predict/Lab2_predict.md

題目:`syscall(462, n)` 要在 kernel 計算 1 + 2 + ⋯ + n,每一步都用 `printk` 印出目前累加值,最後把總和回傳。user 端印出回傳值。以 n = 5 為例。

預期輸出

```
$ ./test
sum(1..5) = 15
$ sudo dmesg | tail -5
[  xxx.xxxxxx] i=1 sum=1
[  xxx.xxxxxx] i=2 sum=3
[  xxx.xxxxxx] i=3 sum=6
[  xxx.xxxxxx] i=4 sum=10
[  xxx.xxxxxx] i=5 sum=15
```

## Lab2_g4 流程/填空/口試題:新增 syscall 要改哪些檔案 猜題 可能性:中

檔案:(無程式碼,見 README) · 出處:review/predict/Lab2_predict.md

題目(筆試或口頭 demo 時問):要新增一個名為 `hello`、編號 462 的 syscall,請寫出:

- (a) 要新增或修改哪些檔案,各加哪一行;
- (b) 從編譯到測試的指令順序;
- (c) `./test` 和 `sudo dmesg | tail` 各會看到什麼。

## Lab2_g5 新增第二個 syscall(編號 463):兩數相加 猜題 可能性:中低

檔案:`hello.c` `test.c` · 出處:review/predict/Lab2_predict.md

題目:保留原本的 hello(462),再新增一個 add syscall(編號 463),接收兩個整數,在 dmesg 印出 `add: 10 + 20 = 30` 並回傳和。

預期輸出

```
$ ./test
hello: 0
add  : 30
$ sudo dmesg | tail -2
[  xxx.xxxxxx] Hello
[  xxx.xxxxxx] add: 10 + 20 = 30
```

## Lab2_g6 參數檢查:不合法時回傳錯誤 猜題 可能性:低

檔案:`hello.c` `test.c` · 出處:review/predict/Lab2_predict.md

題目:沿用 Ex2(印 n 次 hello)。當 n ≤ 0 或 n > 10 時不印任何東西,回傳 `-EINVAL`。user 端分別呼叫 `syscall(462, 3)` 和 `syscall(462, -1)`,失敗時用 `perror` 印出原因。

預期輸出

```
$ ./test
n=3  -> 0
syscall: Invalid argument
n=-1 -> -1
$ sudo dmesg | tail -3
[  xxx.xxxxxx] hello
[  xxx.xxxxxx] hello
[  xxx.xxxxxx] hello
```

`perror` 走 stderr,而且沒有緩衝,所以 terminal 上可能先出現 `syscall: Invalid argument` 再出現 `n=-1 -> -1`;重新導向時順序也可能不同。

# Lab 3:System Call vs APIs

## Lab3_ex 只用 <unistd.h> 印出 integer 課堂 Exercise 可能性:原題

檔案:ex ex.c · 出處:簡報 p.32

- 學習目標:了解為何我們比較偏好使用將 system call 包裝後的函式。
- 要求:在只使用 <unistd.h> 的情況下印出 integer(助教會檢查有沒有偷用 <stdio.h>)。
- 測資:0、100、2147483647、-2147483648。

## Lab3_g1 只用 <unistd.h> 印出字串與整數陣列（含負數） 猜題 可能性:高

檔案:q1.c · 出處:review/predict/Lab3_predict.md

只能 #include <unistd.h>（不可用 <stdio.h>、<string.h>），實作：

- my_puts(const char *s)：印出字串（不自動換行）
- my_print_int(int a)：印出整數（不換行）

用它們印出陣列 {3, -15, 0, 2147483647, -2147483648}（以空白分隔），以及元素個數。

預期輸出

```
arr = 3 -15 0 2147483647 -2147483648
n = 5
```

## Lab3_g2 只用 <unistd.h> 從 stdin（fd 0）讀入兩個整數，輸出和 猜題 可能性:高

檔案:q2.c · 出處:review/predict/Lab3_predict.md

只能 #include <unistd.h>。用 read(0, ...) 從鍵盤讀入兩個整數（可能是負數，中間用空白或換行分隔），輸出兩數相加結果。讀取失敗時把錯誤訊息寫到 stderr（fd 2）。

輸入／輸出範例

```
$ ./q2
Input two integers: 3 -5
sum = -2

$ printf '  -2147483648\n -1\n' | ./q2
Input two integers: sum = -2147483649
```

## Lab3_g3 只用 system call 複製檔案（自製 cp） 猜題 可能性:中高

檔案:q3.c · 出處:review/predict/Lab3_predict.md

寫 mycp，用法 ./mycp <src> <dst>。只能用 open/read/write/close（<unistd.h>、<fcntl.h>），不能用 <stdio.h>。

- 目的檔不存在就建立，權限 0644；已存在則先清空。
- 複製完印出 copied N bytes。
- 來源檔開不起來要印錯誤訊息到 stderr 並 return 1。

輸入／輸出範例

```
$ printf 'Hello World\nline2\n' > src.txt
$ ./mycp src.txt dst.txt
copied 18 bytes
$ cat dst.txt
Hello World
line2
$ ls -l dst.txt
-rw-r--r-- 1 user user 18 ... dst.txt
```

## Lab3_g4 用 open() 指定權限建立檔案，把整數寫進檔案 猜題 可能性:中

檔案:q4.c · 出處:review/predict/Lab3_predict.md

只用 <unistd.h>、<fcntl.h>：建立 score.txt，權限必須是 rw-r-----（owner 讀寫、group 只讀、other 無），把 1~5 的平方一行一個寫進檔案。完成後螢幕印 done，並用 ls -l、cat 給助教看。

輸入／輸出範例

```
$ ./q4
done
$ ls -l score.txt
-rw-r----- 1 user user 12 ... score.txt
$ cat score.txt
1
4
9
16
25
```

（延伸口試題：chmod 0244 score.txt 之後 cat 會怎樣？→ Permission denied，因為 owner 只剩 w，p.12 示範過。）

## Lab3_g5 write() vs fwrite() 一次寫 1 byte，比較時間並用 strace 觀察 猜題 可能性:中

檔案:q5_fwrite.c q5_write.c · 出處:review/predict/Lab3_predict.md

仿照 example4，但改成「寫」：分別用 write() 和 fwrite() 一次寫 1 byte、共寫 1MB（1048576 次）到檔案，印出花費秒數；再用 strace -c 比較 write system call 的次數，並解釋原因。

預期結果

```
$ ./q5_write
0.463326
$ ./q5_fwrite
0.013798
$ strace -c ./q5_write    → write 呼叫 1048577 次
$ strace -c ./q5_fwrite   → write 呼叫 257 次
```

## Lab3_g6 用 C library（fopen/fread/fwrite/fclose）改寫檔案複製 猜題 可能性:低

檔案:q6.c · 出處:review/predict/Lab3_predict.md

把第 3 題的 mycp 改用 fopen/fread/fwrite/fclose 實作，開檔失敗用 perror 印錯誤並 exit(1)。完成後回答：兩個版本用 strace -c 看，誰的 read/write 次數多？

輸入／輸出範例

```
$ ./q6 big.bin d6.bin
copied 10000 bytes
```

# Lab 4:Library

## Lab4_ex Exercise 1 static library + Exercise 2 shared library(sort) 課堂 Exercise 可能性:原題

檔案:`ex1` `ex1.c` `ex_dy` `exlib.a` `exlib.h` `exlib.so` `sort.c` `sort.o` · 出處:簡報 p.14–15、p.26–27

Exercise 1

- `sort.c`:建立 sort 函式,參數接受陣列的位址,將陣列內的整數由小排到大。
- `exercise.c`(本資料夾為 `ex1.c`):建立長度 5 的陣列,放入 5 個不重複隨機整數,先印出 sort 前的值,再呼叫 sort 並印出結果。
- 將 `sort.o` 加入 `exlib.a`,用 gcc 把 exercise.c 與 exlib.a 輸出成 `exercise1`(本資料夾為 `ex1`)。

Exercise 2

- 用同樣兩支程式,改用 shared library 編譯(本資料夾為 `exlib.so`、`ex_dy`),成功後用 `ls -l` 比較檔案大小。

## Lab4_g1 降冪排序 library（10 個不重複亂數） 猜題 可能性:高

檔案:`exercise.c` `exlib.h` `sort.c` · 出處:review/predict/Lab4_predict.md

1. `sort.c`：建立 `sort_desc` 函式，參數接受陣列位址與長度，將陣列由大到小排序。
2. `exercise.c`：產生 10 個不重複、範圍 1~100 的隨機整數，印出排序前／後。
3. 將 `sort.o` 加入 `exlib.a`，編成 `exercise1`（static）。
4. 用 shared library 方式編成 `exercise2`，執行成功後用 `ls -l` 比較檔案大小。

輸出範例：

```
$ ./exercise1
Before sort: 61 67 88 54 4 30 36 8 84 71
After sort: 88 84 71 67 61 54 36 30 8 4
```

## Lab4_g2 統計函式庫 max / min / average（多個 .o 包成一個 library） 猜題 可能性:高

檔案:average.c main.c max.c min.c mylib.h · 出處:review/predict/Lab4_predict.md

1. 撰寫 max.c、min.c、average.c 三個檔案，各自實作一個函式，參數為陣列位址與長度；共用標頭 mylib.h。
2. main.c 產生 5 個不重複隨機整數（1~100），印出陣列、最大值、最小值、平均（小數點後 2 位）。
3. 把三個 .o 包成 mylib.a，編成 stat_static；再包成 mylib.so，編成 stat_dynamic。
4. ls -l 比較兩個執行檔大小，並用 ldd 確認 stat_dynamic 用到 mylib.so。

輸出範例：

```
Array: 94 75 82 29 9
Max = 94
Min = 9
Average = 57.80
```

## Lab4_g3 原題重現 + ldd 驗證 + 刪除 library 實驗 猜題 可能性:中高

檔案:exercise.c exlib.h sort.c · 出處:review/predict/Lab4_predict.md

1. 完成 Exercise 1、Exercise 2（sort 由小到大、5 個不重複亂數，exercise1 / exercise2）。
2. 第一次執行 exercise2 時要出現「找不到 shared library」的錯誤，並修正它。
3. 分別對 exercise1、exercise2 執行 ldd，說明差異。
4. 把 exlib.a、exlib.so（含 /lib 裡那份）移走，再執行兩個程式，說明結果。

輸出範例：

```
$ ./exercise1                ← exlib.a 已移走，仍可執行
Before sort: 60 52 99 95 35
After sort: 35 52 60 95 99
$ ./exercise2                ← /lib/exlib.so 移走後失敗
./exercise2: error while loading shared libraries: exlib.so: cannot open shared object file: No such file or directory
```

## Lab4_g4 更新 library 後觀察 static / dynamic 行為差異 猜題 可能性:中

檔案:`exercise.c` `exlib.h` `sort_v1.c` `sort_v2.c` · 出處:review/predict/Lab4_predict.md

1. 先用 v1 的 `sort.c`（由小到大，排序前印 `[exlib v1] ascending`）建出 `exercise1`（static）和 `exercise2`（dynamic）。
2. 把 `sort.c` 改成 v2（由大到小，印 `[exlib v2] descending`），只重建 `exlib.a`、`exlib.so`，不要重新編譯 exercise.c。
3. 再執行兩個程式，觀察差異並說明原因；最後重新連結 `exercise1` 讓它也變成 v2。

輸出範例（更新 library 後、未重新連結）：

```
$ ./exercise1
Before sort: 34 66 59 54 69
[exlib v1] ascending
After sort: 34 54 59 66 69
$ ./exercise2
Before sort: 34 66 59 54 69
[exlib v2] descending
After sort: 69 66 59 54 34
```

## Lab4_g5 擴充投影片範例 — 四則運算 library 猜題 可能性:中

檔案:`add.c` `div.c` `lib.h` `main.c` `mul.c` `sub.c` · 出處:review/predict/Lab4_predict.md

把 p.5 範例改成兩個參數的四則運算：`add.c`、`sub.c`、`mul.c`、`div.c`（函式 `add / sub / mul / divide`），共用 `lib.h`。`main.c` 讀入兩個整數並印出四則運算結果（除法到小數點後 2 位，除數為 0 要印提示）。分別以 static（`calc_static`）、dynamic（`calc_dynamic`）編譯並 `ls -l`。

輸出範例：

```
Input two integers: 7 2
7 + 2 = 9
7 - 2 = 5
7 * 2 = 14
7 / 2 = 3.50
```

## Lab4_g6 字串函式庫 reverse + to_upper 猜題 可能性:低

檔案:`main.c` `reverse.c` `strlib.h` `upper.c` · 出處:review/predict/Lab4_predict.md

撰寫 `reverse.c`（將字串反轉）與 `upper.c`（將小寫轉大寫），共用 `strlib.h`，打包成 `strlib.a` 與 `strlib.so`。`main.c` 讀入一行字串，依序印出原字串、反轉後、轉大寫後。分別編成 `str_static`、`str_dynamic`，`ls -l` 比較。

輸出範例：

```
Input a string: Hello OS Lab4
Original : Hello OS Lab4
Reverse  : 4baL SO olleH
Upper    : 4BAL SO OLLEH
```

# Lab 5:Kernel Module

## Lab5_ex date2timestamp + myage 課堂 Exercise 可能性:原題

檔案:`Makefile Module.symvers d2ts.c d2ts.ko d2ts.mod d2ts.mod.c d2ts.mod.o d2ts.o modules.order myage.c myage.ko myage.mod myage.mod.c myage.mod.o myage.o` · 出處:簡報 p.31–35

date2timestamp(本資料夾為 `d2ts.c`)

- module parameters:`year`、`month`、`date`;export symbol:`ts`。
- 輸入出生年月日,印出該日期的時間戳記 `ts`,並提供給其他模組使用。
- 若輸入無效日期,顯示錯誤訊息並終止安裝(`printk(KERN_ERR "Invalid argument(s).\n"); return -EINVAL;`)。

myage

- 取得當前時間戳記,與 `ts` 相減,轉成年跟日,輸出你已經活多久;相減小於 0 要提示。
- 提示:`#include <linux/time64.h>`,`mktime64(year, month, date, 0, 0, 0);` 時間戳精度為秒。

(本資料夾的答案多了一個簡報沒要求的 `is_adult()`,是自己練習的延伸。)

## Lab5_g1 倒數計時 猜題 可能性:高

檔案:`Makefile countdown.c deadline.c` · 出處:review/predict/Lab5_predict.md

題目敘述 請撰寫兩個 kernel module：

- `deadline`：module parameters `year`、`month`、`date`（預設 2026/12/25），export symbol `ts`。印出該日期的時間戳記；若日期無效，印出錯誤訊息並終止安裝。
- `countdown`：取得當前時間戳記，算出距離 deadline 還剩幾天幾時幾分幾秒；若該日期已經過了，要提示「已經過了幾天」。

```
sudo insmod deadline.ko year=2026 month=2 date=29     # 2026 不是閏年 → 失敗
sudo insmod deadline.ko year=2026 month=12 date=25
sudo insmod countdown.ko
sudo dmesg | tail -4
```

預期輸出：

```
insmod: ERROR: could not insert module deadline.ko: Invalid parameters
[...] Invalid argument(s).
[...] deadline insert successfully! 2026/12/25 ts=1798156800
[...] countdown insert successfully!
[...] There are 57 days 22 hours 0 minutes 0 seconds left!
```

日期已過（`year=2026 month=10 date=1`）時：`The target date has passed 27 days 2 hours ago!`

## Lab5_g2 加上時分秒：活了幾年幾天幾時幾分幾秒 猜題 可能性:高

檔案:Makefile date2timestamp.c myage.c · 出處:review/predict/Lab5_predict.md

題目敘述

- date2timestamp：參數改成 year month date hour minute second 六個（hour 0~23、minute／second 0~59），超出範圍視為無效 → 錯誤訊息＋終止安裝；export ts。
- myage：輸出你活了「X 年 Y 天 Z 時 W 分 S 秒」（一年以 365 天計）；若 ts 在未來要提示。

```
sudo insmod date2timestamp.ko year=2001 month=8 date=1 hour=24            # hour 超出範圍
sudo insmod date2timestamp.ko year=2001 month=8 date=1 hour=13 minute=30 second=5
sudo insmod myage.ko
sudo dmesg | tail -4
```

預期輸出：

```
insmod: ERROR: could not insert module date2timestamp.ko: Invalid parameters
[...] Invalid argument(s).
[...] date2timestamp insert successfully! ts=996672605
[...] myage insert successfully!
[...] You have lived 25 years 93 days 12 hours 29 minutes 55 seconds!
```

ts 在未來（例如 year=2026 month=10 date=28 hour=3）：You are not born yet!（diff=-3600）

## Lab5_g3 Export 函式：兩個日期相差幾天 猜題 可能性:中高

檔案:Makefile datelib.c daydiff.c · 出處:review/predict/Lab5_predict.md

題目敘述

- datelib：不需要參數。export 兩個函式：int is_valid_date(int y, int m, int d)（有效回傳 1，否則 0）、time64_t date2ts(int y, int m, int d)（回傳時間戳記）。
- daydiff：參數 year1 month1 date1 year2 month2 date2，呼叫 datelib 的函式驗證兩個日期（任一無效 → 錯誤訊息＋終止安裝），印出兩個日期相差幾天（日期 2 早於日期 1 時為負數）。

```
sudo insmod daydiff.ko                      # 先裝 daydiff → 失敗
sudo insmod datelib.ko
sudo insmod daydiff.ko year1=2001 month1=8 date1=1 year2=2026 month2=10 date2=28
sudo dmesg | tail -3
sudo rmmod datelib                          # 故意先拆 datelib → 失敗
```

預期輸出：

```
insmod: ERROR: could not insert module daydiff.ko: Unknown symbol in module
[...] datelib insert successfully!
[...] daydiff insert successfully!
[...] From 2001/8/1 to 2026/10/28: 9219 days
rmmod: ERROR: Module datelib is in use by: daydiff
```

其他測資：2024/1/1 → 2025/1/1：366 days（閏年）、2026/12/25 → 2026/10/28：-58 days、year1=2025 month1=2 date1=29 → Invalid argument(s).

## Lab5_g4 精確足歲＋下次生日倒數 猜題 可能性:中

檔案:Makefile birthday.c date2timestamp.c · 出處:review/predict/Lab5_predict.md

題目敘述 exercise 的 myage 用「365 天＝1 年」近似，會因閏日而偏差。請改寫：

- date2timestamp：同原題（參數 year/month/date、export ts），但另外把 year、month、date 也 export。
- birthday：算出真正的「足歲」與「距離上次生日幾天」，以及「下次生日還有幾天幾小時」。ts 在未來時要提示。只能用 mktime64。

```
sudo insmod date2timestamp.ko year=2001 month=8 date=1
sudo insmod birthday.ko
sudo dmesg | tail -3
```

預期輸出：

```
[...] birthday insert successfully!
[...] You are 25 years old, 88 days since your last birthday.
[...] Your next birthday is in 276 days 22 hours.
```

對照：原本 myage（365 天近似）會印 25 years and 94 days，差的 6 天就是 2004、2008、2012、2016、2020、2024 這 6 個閏日。 其他測資：2005/10/29 → 20 years old, 364 days since…、next birthday is in 0 days 22 hours；2004/2/29 → 22 years old, 241 days、next … 123 days 22 hours（平年把 3/1 當生日）。

## Lab5_g5 出生那天是星期幾 猜題 可能性:中低

檔案:Makefile date2timestamp.c weekday.c · 出處:review/predict/Lab5_predict.md

題目敘述

- date2timestamp：同原題。
- weekday：用 ts 算出生日是星期幾（已知 1970/1/1 是星期四），並印出活了幾天、合幾週又幾天。

```
sudo insmod date2timestamp.ko year=2001 month=8 date=1
sudo insmod weekday.ko
sudo dmesg | tail -3
```

預期輸出：

```
[...] weekday insert successfully!
[...] You were born on Wednesday.
[...] You have lived 9219 days = 1317 weeks and 0 days.
```

year=1970 month=1 date=1 → Thursday、20754 days = 2964 weeks and 6 days。（今天 2026/10/28 是星期三，和 2001/8/1 同星期，所以剛好整週。）

## Lab5_g6 單一 module：該年第幾天、是否閏年 猜題 可能性:低

檔案:`Makefile` `dayofyear.c` · 出處:review/predict/Lab5_predict.md

題目敘述 撰寫一個 kernel module dayofyear，參數 `year month date`。若日期無效，顯示錯誤訊息並終止安裝；否則印出該日期是當年的第幾天、該年是否閏年、今年還剩幾天。

```
sudo insmod dayofyear.ko year=2100 month=2 date=29     # 2100 不是閏年
sudo insmod dayofyear.ko year=2024 month=8 date=1
sudo dmesg | tail -3
modinfo dayofyear.ko | grep parm
```

預期輸出：

```
insmod: ERROR: could not insert module dayofyear.ko: Invalid parameters
[...] Invalid argument(s).
[...] dayofyear insert successfully!
[...] 2024/8/1 is day 214 of 2024 (a leap year), 152 days left.
parm:           year:year (int)
parm:           month:month (int)
parm:           date:date (int)
```

其他測資：`2000/2/29` → `day 60 … (a leap year), 306 days left`；`2026/10/28` → `day 301 … (not a leap year), 64 days left`。

# Lab 6:Process Control

## Lab6_ex1 Exercise 1(40 分):fork 後 global / local 變數 課堂 Exercise 可能性:原題

檔案:ex1 ex1.c · 出處:簡報 p.22–24

- 宣告一個 global 變數和一個 local 變數,初值都是 10。
- 在螢幕上印出兩個變數的值(附 process id 與 parent 的 id)。
- 建立一個 child process,印出兩個變數,然後把兩個變數遞增。
- 最後兩個 process 都印出 global 與 local 的值,附自己的 PID 與 PPID。
- (對照 p.24 Result:只有 child 遞增,parent 印出的仍是 10。)

## Lab6_ex2 Exercise 2(50 分):5 個 process 的樹 課堂 Exercise 可能性:原題

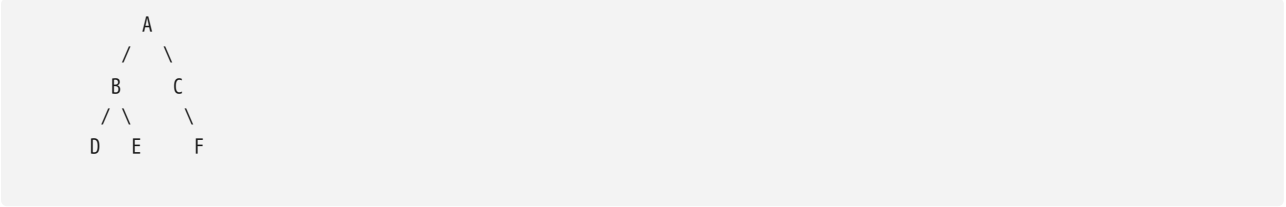
檔案:ex2 ex2.c · 出處:簡報 p.25–27

- 寫一個程式建立 5 個 process,形成指定的樹狀結構(A → B、D、E;B → C)。
- 每個 process 印出自己的 process id 與 parent 的 process id。
- 有 child 的 process 必須依 child 的數目呼叫相同次數的 `wait()`,等所有 child 結束。

## Lab6_g1 換一棵 process tree（6 個 process） 猜題 可能性:高

檔案:q1_tree.c · 出處:review/predict/Lab6_predict.md

題目：寫一個程式建立 6 個 process，形成下面這棵樹。每個 process 印出自己的 PID 和 parent 的 PID。有 child 的 process 必須依照 child 的數目呼叫相同次數的 `wait()`，等所有 child 結束後才印自己的資料。




輸出範例（PID 會不同，順序可能不同，但 parent 一定在自己所有 child 之後）：

```
Child process D -   PID: 1891     , PPID: 1889
Child process E -   PID: 1892     , PPID: 1889
Child process F -   PID: 1893     , PPID: 1890
Child process B -   PID: 1889     , PPID: 1888
Child process C -   PID: 1890     , PPID: 1888
Parent process A -  PID: 1888     , PPID: 1887
```

## Lab6_g2 Ex1 改成 parent 和 child 都遞增，加上 heap 變數 猜題 可能性:高

檔案:`q2_vars.c` · 出處:review/predict/Lab6_predict.md

題目：宣告一個 global 變數、一個 local 變數，以及一個用 `malloc` 配置的 heap 變數，初值都是 10，先印一次（附 PID、PPID）。fork 後：

- child 先印一次，然後三個變數各 +1，再印一次。
- parent 用 `wait()` 等 child 結束，印一次，然後三個變數各 +5，再印一次。
- （加分）兩個 process 都印出三個變數的位址。

輸出範例：

```
Initial value:
PID: 2009     ,PPID: 1985     , Global: 10   , Local: 10   , Heap: 10
---------------------------------------------------------------
Child process:
PID: 2010     ,PPID: 2009     , Global: 10   , Local: 10   , Heap: 10
Child process after increment:
PID: 2010     ,PPID: 2009     , Global: 11   , Local: 11   , Heap: 11
Child  &global=0x55cbe1fdf010  &local=0x7fffdc8a4528  heap=0x55cbfa0352a0
Parent process:
PID: 2009     ,PPID: 1985     , Global: 10   , Local: 10   , Heap: 10
Parent process after increment:
PID: 2009     ,PPID: 1985     , Global: 15   , Local: 15   , Heap: 15
Parent &global=0x55cbe1fdf010  &local=0x7fffdc8a4528  heap=0x55cbfa0352a0
---------------------------------------------------------------
```

## Lab6_g3 fork–exec，child 用 execlp 執行 ls -l，parent 等待後印訊息 猜題 可能性:中高

檔案:`q3_exec.c` · 出處:review/predict/Lab6_predict.md

題目：parent fork 出一個 child。child 印出自己的 PID、PPID，然後用 `execlp` 執行 `ls -l`。parent 用 `wait()` 等 child 結束後，印出自己的 PID、child 的 PID，以及 `wait()` 的回傳值。

輸出範例：

```
Parent start: PID = 2116
Child: PID = 2118, PPID = 2116, now exec "ls -l"
total 0
-rw-r--r-- 1 root root 0 Oct 27 12:02 a.txt
-rw-r--r-- 1 root root 0 Oct 27 12:02 b.c

------ this is parent process ------
Pid : 2116  child pid : 2118    wait() returned : 2118
-------------------------------------
```

## Lab6_g4 用迴圈 fork N 個 child（扇形樹），parent wait N 次並印出回收的 child PID 猜題

可能性:中

檔案:`q4_fan.c` · 出處:review/predict/Lab6_predict.md

題目：parent 用 for 迴圈產生 3 個 child。每個 child 印出「I am child process i.」以及 PID、PPID 後結束。parent 呼叫 3 次 `wait()`，每次印出結束的是哪一個 child（wait 的回傳值），最後印出自己的資料。

```
        P
      / | \
    C1  C2  C3
```

輸出範例：

```
I am child process 2.   Pid: 2135   Parent Pid: 2133
I am child process 1.   Pid: 2134   Parent Pid: 2133
I am child process 3.   Pid: 2136   Parent Pid: 2133
Parent: child 2134 terminated
Parent: child 2135 terminated
Parent: child 2136 terminated
I'm parent process. Pid: 2133   Parent Pid: 2090
```

## Lab6_g5 鏈狀 process tree A → B → C → D 猜題 可能性:中

檔案:`q5_chain.c` · 出處:review/predict/Lab6_predict.md

題目：建立 4 個 process，形成一條鏈：A 產生 B，B 產生 C，C 產生 D。每個 process 都要先 wait 自己的 child，再印出自己的名稱、PID、PPID。

```
A
 └─── B
     └─── C
         └─── D
```

輸出範例（一定是由下往上印）：

```
Process D - PID: 1915     , PPID: 1914
Process C - PID: 1914     , PPID: 1913
Process B - PID: 1913     , PPID: 1912
Process A - PID: 1912     , PPID: 1887
```

## Lab6_g6 fork() 連續呼叫 n 次，總共有幾個 process？ 猜題 可能性:低～中

檔案:`q6_count.c` · 出處:review/predict/Lab6_predict.md

題目：執行下面的程式，回答總共會印出幾行？畫出 process tree。（或者反過來：寫程式讓每個 process 都印出 PID/PPID，再數一數總共有幾個。）

```
for (i = 0; i < 3; i++)
    fork();
printf("PID: %-8d , PPID: %d\n", getpid(), getppid());
```

答案：每呼叫一次 fork，process 數就乘以 2，所以總共 $2^3 = 8$ 個 process，印 8 行。樹的形狀（P0 是原始 process）：

```
P0 ─┬─ P1 ─┬─ P3 ── P7
    │      └─ P4
    ├─ P2 ── P5
    └─ P6
 (P0 有 3 個 child, 第一個 child 有 2 個, 依此類推：3, 2, 1, 1, 0, 0, 0, 0)
```

# Lab 7:IPC(Message Queue / Shared Memory)

## Lab7_ex1 Exercise-Lab1(50 分):Message Queue sender / receiver 課堂 Exercise 可能性:原題

檔案:`receiver` `receiver.c` `sender` `sender.c` · 出處:Lab7-1.pdf p.22–24

- `sender.c`:建立 message queue;使用者可反覆輸入字串,直到輸入 `"exit"`;把訊息送到 message queue。
- `receiver.c`:反覆從 message queue 接收訊息並顯示在螢幕上;最後 close & unlink message queue。
- 可能用到:`fgets(buf, BUFSIZ, stdin)`、`strcmp(message, "hello")`。

## Lab7_ex2 Exercise 2(50 分):Shared Memory producer / consumer 課堂 Exercise 可能性:原題

檔案:`consumer` `consumer.c` `producer` `producer.c` · 出處:Lab7-2.pdf p.26–28

- `producer.c`:建立 shared memory;使用者輸入訊息寫進 shared memory。
- `consumer.c`:把 shared memory 的內容印在螢幕上。
- 編譯要加 `-lrt`。

## Lab7_g1 雙向 Message Queue：計算機 client / server 猜題 可能性:高

檔案:`client.c` `server.c` · 出處:review/predict/Lab7_predict.md

寫兩支程式：

- `client.c`：建立兩個 message queue（請求用、回應用）。使用者反覆輸入 `a op b`（op 為 `+ - * /`），送給 server，然後等 server 回傳結果並印出；輸入 `exit` 時把 `exit` 也送給 server，然後結束。
- `server.c`：反覆接收運算式，計算後把結果送回 client，並在自己的畫面印出；收到 `exit` 時結束，close 並 unlink 兩個 queue。

輸出範例（終端機 A：server 先執行）：

```
$ ./server
Server is waiting...
received: 3 + 5 -> 8
received: 10 * 4 -> 40
received: 7 / 0 -> invalid input
received: 100 - 1 -> 99
Server exit.
```

終端機 B：

```
$ ./client
Enter expression (e.g. 3 + 5), type exit to quit:
> 3 + 5
result: 8
> 10 * 4
result: 40
> 7 / 0
result: invalid input
> 100 - 1
result: 99
> exit
```

## Lab7_g2 Shared Memory 寫入多筆訊息直到 exit 猜題 可能性:高

檔案:`consumer.c` `producer.c` · 出處:review/predict/Lab7_predict.md

- `producer.c`：建立 shared memory，讓使用者反覆輸入訊息直到 `exit`（最多 10 筆），全部存進 shared memory，結束時印出寫入筆數。
- `consumer.c`：讀出 shared memory 中所有訊息，加上編號印出，最後 `munmap`、`close`、`shm_unlink`。

終端機 A：

```
$ ./producer
Enter messages (type exit to finish):
> hello
> OS lab 7
> shared memory
> exit
3 message(s) written to shared memory.
```

終端機 B（producer 結束後執行）：

```
$ ./consumer
Read 3 message(s) from shared memory:
[1] hello
[2] OS lab 7
[3] shared memory
```

## Lab7_g3 Shared Memory 傳整數陣列，consumer 計算 猜題 可能性:中高

檔案:`consumer.c` `producer.c` · 出處:review/predict/Lab7_predict.md

- `producer.c`：使用者反覆輸入整數直到 `exit`，把整數（不是字串）存進 shared memory。
- `consumer.c`：讀出所有整數，印出 sum、avg（小數兩位）、max、min，最後清理 shared memory。

終端機 A：

```
$ ./producer
Enter integers (type exit to finish):
> 12
> 7
> -3
> 25
> 9
> exit
5 number(s) written.
```

終端機 B：

```
$ ./consumer
Numbers: 12 7 -3 25 9
sum = 50, avg = 10.00, max = 25, min = -3
```

## Lab7_g4 依 Priority 接收 + mq_getattr + O_NONBLOCK 猜題 可能性:中

檔案:`receiver.c` `sender.c` · 出處:review/predict/Lab7_predict.md

- `sender.c`：使用者反覆輸入 `<priority> <message>` 直到 `exit`，以指定 priority 送進 queue；結束時只 close 不 unlink。
- `receiver.c`：在 sender 結束之後才執行。先用 `mq_getattr` 印出 queue 的 `maxmsg / msgsize / curmsgs`，再用非阻塞方式把 queue 中訊息全部收完並印出各自的 priority，queue 空了就結束，最後 close & unlink。

終端機 A：

```
$ ./sender
Input format: <priority> <message>, type exit to quit
> 1 low task
> 5 urgent task
> 3 normal task A
> 3 normal task B
> 9 fire alarm
> exit
```

終端機 B（之後執行）：

```
$ ./receiver
maxmsg=10, msgsize=256, curmsgs=5
received (prio 9): fire alarm
received (prio 5): urgent task
received (prio 3): normal task A
received (prio 3): normal task B
received (prio 1): low task
queue is empty.
```

## Lab7_g5 結合兩者：shm 放資料、message queue 當通知 猜題 可能性:中

檔案:`consumer.c` `producer.c` · 出處:review/predict/Lab7_predict.md

Exercise 2 的 consumer 必須等 producer 寫完才能手動執行。請改成：consumer 可以先執行並等待，producer 每輸入一筆訊息就寫進 shared memory，然後用 message queue 通知 consumer「資料好了」；consumer 收到通知才去讀 shared memory 並印出，讀完再回覆 producer 可以寫下一筆。輸入 `exit` 時兩邊都結束，由 consumer 清除所有 IPC 物件。

終端機 A（consumer 先執行）：

```
$ ./consumer
Waiting for producer...
Read from shared memory: hello
Read from shared memory: second message
Read from shared memory: bye bye
Producer finished.
```

終端機 B：

```
$ ./producer
Enter the message: hello
Enter the message: second message
Enter the message: bye bye
Enter the message: exit
```

## Lab7_g6 驗證觀念：kernel persistence 與 MAP_PRIVATE vs MAP_SHARED 猜題 可能性:中低

檔案:`cleaner.c` `reader.c` `writer.c` · 出處:review/predict/Lab7_predict.md

1. `writer.c`：建立 shared memory，寫入 `"original data"` 後直接結束（不 unlink）。用 `ls /dev/shm` 證明 process 結束後物件仍存在。
2. `reader.c`：先用 `MAP_PRIVATE`（PROT_READ|PROT_WRITE）映射並修改內容，再用 `MAP_SHARED` 映射同一個物件，印出內容，證明 MAP_PRIVATE 的修改不會影響到 shared memory；接著用 MAP_SHARED 修改。再執行一次 reader 證明 MAP_SHARED 的修改有留下來。
3. `cleaner.c`：印出最終內容並 `shm_unlink`，之後 `ls /dev/shm` 看不到物件。

輸出範例：

```
$ ./writer
writer: wrote "original data" and exit (no shm_unlink)
$ ls /dev/shm
persistShm_lab7p
$ ./reader
[PRIVATE] before: original data
[PRIVATE] after : modified by MAP_PRIVATE
[SHARED ] content: original data
[SHARED ] after : modified by MAP_SHARED
$ ./reader
[PRIVATE] before: modified by MAP_SHARED
[PRIVATE] after : modified by MAP_PRIVATE
[SHARED ] content: modified by MAP_SHARED
[SHARED ] after : modified by MAP_SHARED
$ ./cleaner
cleaner: final content = modified by MAP_SHARED
cleaner: shm_unlink done
$ ls /dev/shm
```